Arquitetura de Software: Teoria e Prática 2026

Felipe Melchior de Britto | Igor Benedito Costa Gonçalves | Sérgio Leandro Alves da Silva

Acionamento de Ambulância com reserva de Leito

A regulação de urgência combina dois recursos escassos e disputados ao mesmo tempo: a ambulância que sai ao local e o leito que precisa existir no destino. O fluxo atravessa contextos distintos — ocorrência, frota, leito hospitalar e notificação — que não podem compartilhar a mesma transação nem o mesmo ciclo de atendimento. Uma central monolítica acopla despacho, ocupação de leito e aviso ao hospital; uma divisão em serviços sem coordenação gera estados inválidos: ambulância em deslocamento sem leito; leito reservado em duplicidade para duas ocorrências; chamado aberto após a equipe já ter sido liberada; aviso que não chega à unidade de destino.

O problema de arquitetura é a coordenação transacional distribuída em um fluxo de baixa latência. Sem orquestração e compensação, falhas parciais (hospital indisponível, recusa de leito, timeout da equipe) deixam frota e leito inconsistentes. Consultas contínuas de ambulâncias livres e de leitos por especialidade, se forem sempre ao banco, degradam o tempo de despacho; cache sem invalidação faz a regulação escolher um recurso que já não existe. Sistemas externos (prontuário da unidade, canal de alerta) falham; sem resiliência no roteamento, a falha da unidade se propaga para a central e o chamado fica sem resposta. O estudo de caso é uma plataforma própria de despacho e reserva de leito, no recorte de um consórcio municipal de urgência. Não se pretende um SAMU completo (geolocalização em tempo real, protocolo clínico, faturamento). O objetivo é tornar observável e tratável a inconsistência entre ocorrência, frota e leito em uma arquitetura de microsserviços.

Descrição da(s) solução(ões)

A solução é uma arquitetura de microsserviços com orquestrador de Saga no despacho, mensageria para efeitos colaterais, cache da disponibilidade de frota e leitos, e API Gateway com políticas de resiliência. O sistema de interesse é a Plataforma de Despacho de Ocorrências e Reserva de Leitos. O sistema superior é a rede municipal de urgência (central de regulação, bases de ambulância, unidades hospitalares e canais de alerta). Os sistemas subordinados (subsistemas): Identidade, Ocorrencias, Frota, Leitos, Notificacoes. Os atores que interagem: Regulador (central), Equipe da ambulância, Unidade hospitalar, Canal de alerta e API Gateway. Fluxo de despacho: (1) registrar ocorrência ABERTA e classificar a prioridade; (2) alocar ambulância DISPONIVEL; (3) reservar leito compatível na unidade de destino; (4) em sucesso, marcar ocorrência DESPACHADA, leito RESERVADO e publicar notificação à equipe e ao hospital; (5) em falha ou timeout, compensar (liberar ambulância, cancelar reserva de leito, reenfileirar o chamado ou cancelá-lo). Reservas de leito não confirmadas pela unidade após X minutos expiram por job agendado, reutilizando as mesmas compensações. A entrega das mensagens será at-least-once, com operações idempotentes (chave da ocorrência)

para evitar dupla alocação de ambulância ou de leito.

Estudo de caso a ser desenvolvido: abertura de um chamado de urgência autenticado, duas ambulâncias e um conjunto limitado de leitos (com e sem vaga), hospital simulando aceite e recusa, e inspeção do estado final em ocorrência, frota, leito e notificação. Relatórios gerenciais, GIS e envio real de SMS ficam mockados. A interface será de menor esforço (OpenAPI/Swagger).

- Membro 1 — API Gateway, roteamento e resiliência (Resilience4j: timeout, retry, circuit breaker). Funcionalidade: autenticar regulador e abrir ocorrência.
- Membro 2 — Mensageria e padrão Saga (orquestração ocorrência–ambulância–leito). Funcionalidade: despachar ambulância e reservar leito, com compensação.
- Membro 3 — Cache de disponibilidade e agendamento. Funcionalidade: consultar/atualizar ocupação e expirar reservas não confirmadas.

Descrição da(s) tecnologia(s)

- Java 21 e Spring Boot (Maven): ecossistema obrigatório; base dos tutoriais hands-on (JPA, MongoDB, REST, OpenAPI, JUnit 5, Docker).
- Spring Web + springdoc-openapi/Swagger: APIs REST e demonstração das funcionalidades (menor esforço de UI).
- Spring Data JPA + MySQL: persistência de Paciente, Ocorrencia, Ambulancia, UnidadeSaude, Leito e Despacho. Resolve o armazenamento transacional local de cada serviço.
- Spring Data MongoDB: histórico de Notificação e trilha de Auditoria do despacho. Resolve a separação do contexto de eventos em relação ao modelo relacional.
- Spring AMQP (RabbitMQ): filas de despacho, recusa de leito e alerta. Resolve o desacoplamento temporal dos passos da Saga.
- Spring Data Redis: cache de ambulâncias livres e de leitos por especialidade. Resolve a latência das consultas da regulação, com invalidação após alocar, confirmar ou compensar.
- Spring Cloud Gateway: ponto único de entrada da central. Resolve a composição da plataforma sem expor endereços internos à regulação e ao hospital.
- Resilience4j: tolerância a falhas na chamada ao MS-Leitos (unidade simulada indisponível). Resolve a propagação da instabilidade hospitalar para a central.
- Spring Scheduler: expiração de reservas não confirmadas. Resolve a devolução automática de leito e ambulância sem orquestrador externo.
- JUnit 5: testes de alocação exclusiva, idempotência e compensação.